

1-Question: What is the purpose of the finally block?

A: The `finally` block ensures that a specific section of code is executed regardless of whether an exception is thrown or caught. It is typically used for cleanup operations, like closing database connections or files, ensuring resources are safely released.

2-Question: How does `int.TryParse()` improve program robustness compared to `int.Parse()`?

A: `int.Parse()` will crash the program (throw a `FormatException`) if the user enters invalid characters. `int.TryParse()` improves robustness because it handles invalid input safely by returning a boolean (`false`) and assigning a default value (`0`), preventing runtime crashes without needing heavy try-catch blocks.

3-Question: What exception occurs when trying to access `Value` on a null `Nullable`?

A: If you attempt to read the `.Value` property of a nullable type when it is currently null, the program will throw an `InvalidOperationException`.

4-Question: Why is it necessary to check array bounds before accessing elements?

A: Arrays have a fixed size allocated in memory. Checking the bounds prevents the program from attempting to read or write data outside that allocated memory, which would result in an `IndexOutOfRangeException` and crash the application.

5-Question: How is the `GetLength(dimension)` method used in multi-dimensional arrays?

A: It is used to retrieve the exact number of elements in a specific dimension of a multi-dimensional array. For example, `Matrix.GetLength(0)` gets the number of rows, and `Matrix.GetLength(1)` gets the number of columns.

6-Question: How does the memory allocation differ between jagged arrays and rectangular arrays?

A: Rectangular Arrays (`int[,]`) allocate memory as a single continuous rectangular block in the Heap. Jagged Arrays (`int[][]`) allocate memory as an "array of arrays," meaning the main array holds references (pointers) to several sub-arrays scattered dynamically in different locations across the Heap, allowing each row to have a different length.

7-Question: What is the purpose of nullable reference types in C#?

A: Introduced in C# 8.0, nullable reference types force the developer to explicitly express intent (`string?`) if a variable is allowed to be null. This helps the compiler analyze the code and show warnings, heavily reducing the chances of a fatal `NullReferenceException` at runtime.

8-Question: What is the performance impact of boxing and unboxing in C#?

A: Boxing and unboxing significantly degrade performance. Boxing forces a value type to be wrapped inside an object and moved from the fast Stack memory to the slower Heap memory. Unboxing requires runtime type-checking and memory extraction. This process consumes CPU time and increases pressure on the Garbage Collector.

9-Question: Why must out parameters be initialized inside the method?

A: The C# compiler enforces this to guarantee that a valid value is assigned before the method finishes execution. This prevents the calling code from receiving uninitialized variables, which could lead to undefined behavior or logical errors.

10-Question: Why must optional parameters always appear at the end of a method's parameter list?

A: To prevent ambiguity. When calling a method, C# maps the arguments you provide to the parameters based on their position from left to right. If optional parameters were at the beginning, the compiler wouldn't know if you are intentionally skipping the optional parameter or trying to assign a value to it.

11-Question: How does the null propagation operator prevent `NullReferenceException`?

A: The null propagation operator (`?.`) automatically checks if the object is null before attempting to access its properties or methods. If the object is null, it immediately stops execution of that line and returns null, rather than throwing a crash-inducing `NullReferenceException`.

12-Question: When is a switch expression preferred over a traditional if statement?

A: Switch expressions are preferred when you need to match a single variable against multiple exact values to return or assign a result. They are cleaner, require far less syntax, and enforce "exhaustiveness" (ensuring all possible cases, including a default `_`, are handled).

13-Question: What are the limitations of the `params` keyword in method definitions?

A:

It must be a single-dimensional array.

It must be the very last parameter in the method signature.

A method can only have one `params` keyword.

